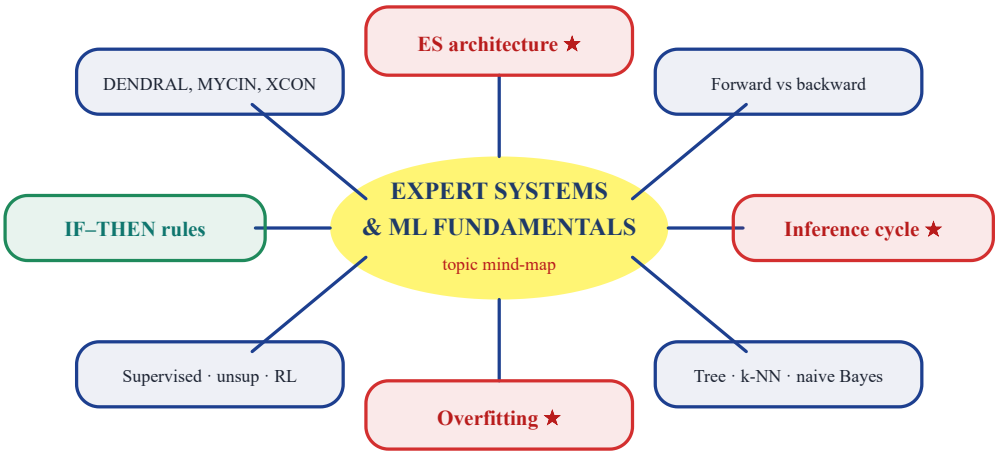
Expert Systems & Machine Learning

knowledge base · inference engine · forward & backward chaining · DENDRAL, MYCIN, XCON · overfitting and bias–variance: 12 sheets



↑ the whole topic on one page: red = high-yield, green = the IF–THEN rule everything hangs on

Syllabus coverage

The Artificial Intelligence syllabus lists "**Introduction to expert system**" and "**Introduction to machine learning; learning algorithms: supervised and clustering (unsupervised)**". This notebook covers exactly those two lines. Search, probabilistic reasoning and NLP are the neighbouring lessons; regression and practical applications come next.

High-yield list

① KB + IE + UI + EF
② match → fire → update → test
③ forward: facts push
④ backward: goal pulls
⑤ DENDRAL, MYCIN, XCON
⑥ labels ⇒ supervised
⑦ 60 / 20 / 20 split
⑧ train ≪ test ⇒ overfitting

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

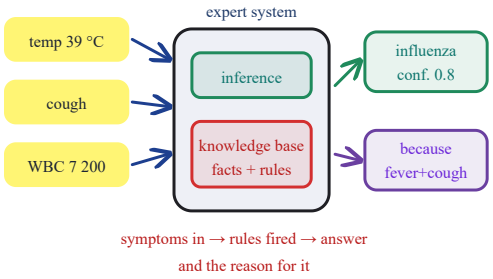
Contents

✓ What is an expert system … p. 2
✓ Architecture of an expert system … p. 3
✓ Rules, cycle & chaining … p. 4
✓ DENDRAL, MYCIN, XCON … p. 5
✓ ML and the three paradigms … p. 6
✓ Supervised learning … p. 7
✓ Split & overfitting … p. 8
✓ Evaluating a model … p. 9
✓ Tree · k-NN · naive Bayes … p. 10
✓ Night-before p. 11 · Practice p. 12

1 What is an Expert System?

Definition: An **expert system** is a program that reproduces the reasoning of a human expert inside one narrow **domain**. It stores that domain's facts and rules, reasons over them to advise, diagnose or recommend, and — crucially — **explains how it reached the conclusion**.

> knowledge-based · rule-based · symbolic



the two outputs that matter: the conclusion, and its justification

The domain must be narrow enough to be **written down**: bacteraemia, tax computation, one family of computers. It need not be easy — a hard domain is fine as long as every judgement can be stated. What disqualifies a domain is not difficulty but **tacit knowledge**: everything the expert does without noticing.

Characteristic	What it means	Example
Domain-specific	one narrow field only	bacterial infection
Rule-based	production rules IF–THEN	IF fever > 38 °C THEN flu
Uncertainty	certainty factors, fuzzy logic	MYCIN's certainty factors
Self-explaining	prints the rules that fired	"because X and Y ⇒ Z"
Does not learn	rules hand-encoded by people	nothing fitted at run time

Memory trick: Four verbs, four parts: it **stores** what it knows, **works out** what follows, **explains** why, and **asks you** for the rest.

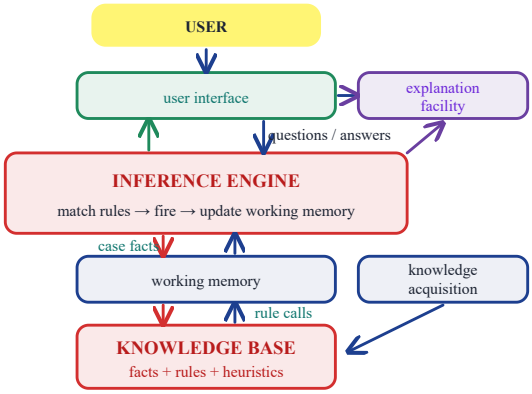
Exam trap: It is **not** a learning system — nothing is learnt from data while it runs. It is also **not** general purpose: narrowness is the design.

Worked example ✍ one rule base, three cycles: **R1** IF temperature > 38 °C AND cough AND white-cell count > 6000 THEN influenza (0.8) · **R2** IF influenza AND shortness of breath THEN pneumonia (0.9) · **R3** IF temperature > 38 °C AND rash AND no cough THEN measles (0.7).
Working memory after the dialogue: temperature 39, cough true, white-cell count 7200, shortness of breath true. Cycle 1: only R1 matches → fire → add *influenza 0.8*. Cycle 2: R2 now matches → fire → add *pneumonia 0.9*. Cycle 3: R3 fails (no rash, cough present). Output: **pneumonia**, explained as "influenza, from fever, cough and raised white cells, plus shortness of breath". One rule fired per cycle — that is all of expert-system reasoning.

IT link: Every loan-scoring rule, tax bracket and fraud rule in production is a miniature expert system: a store of rules, an engine that matches them, a log that explains it — a **policy engine**.

2 Architecture of an Expert System

Definition: The **knowledge base** stores the domain's facts, rules and heuristics. The **inference engine** matches rules against what is known and fires the ones that match. The **working memory** holds the facts of the case in hand. The **user interface** is the dialogue. The **explanation facility** prints the reasoning.



the five boxes of every expert system, plus the sixth that feeds it

Component	Job	The question it answers
Knowledge base	facts + rules + heuristics	"what is known?"
Inference engine	matches, fires, updates	"what follows from it?"
Working memory	facts of this case	"what do we know now?"
User interface	asks, takes answers	"what do you need?"
Explanation facility	traces the rules fired	"why did you say that?"
Knowledge acquisition	elicits and adds rules	"how does a rule arrive?"

Memory trick: Ask the four questions in order — **what** is known (knowledge base), **so what** follows (inference engine), **why** (explanation), **what else** do I need (interface).

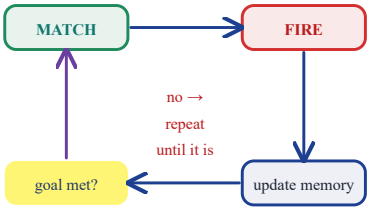
Exam trap: The inference engine **does not store** the rules — the knowledge base does; the engine only matches and fires. The explanation facility is **not** the user interface.

Worked example ✍ which part answers which question?: (a) "Is this chest pain a heart attack?" — asked by the **user interface**, decided by the **inference engine**. (b) "Why do you think so?" — the **explanation facility**. (c) "How much do you weigh?" — interface, and the answer goes into **working memory**. (d) "Rules 1 and 4 both match; which wins?" — conflict resolution inside the engine.

IT link: The same five boxes appear in every config-driven system: a router's access-list matcher, a tax engine, a spam rule list — the explanation facility replaced by an audit log.

3 Rules, the Cycle & the Two Chainings

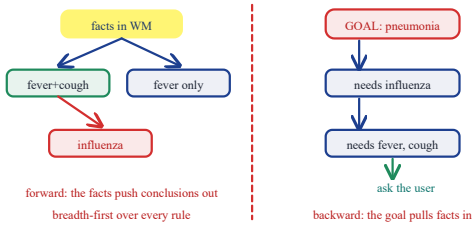
Definition: A production rule reads **IF <antecedent> THEN <consequent>**, the antecedent being a conjunction of conditions on working-memory facts. The **inference cycle** is **match → fire → update → test**: match every rule against working memory, fire the ones that match, copy each conclusion into working memory, then test the goal.



one turn of the cycle

Part of a rule	Example
Antecedent (IF)	temperature > 38 °C
Connectives	AND / OR / NOT
Consequent (THEN)	influenza, CF 0.8
Certainty factor	MYCIN: −1 … +1
Firing	copy the THEN into memory

Feature	Forward chaining — data-driven	Backward chaining — goal-driven
Starts from	the facts in working memory	a goal / hypothesis
Search style	breadth-first, bottom-up	depth-first, top-down
Answers	"what else can be derived?"	"is this goal provable?"
Best for	monitoring, prediction	diagnosis, planning
Weakness	explores every rule	no rule for the goal
Classic user	XCON (configuration)	MYCIN (diagnosis)



same rule base, two directions of travel

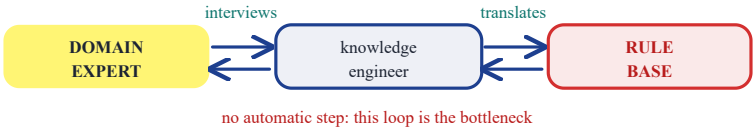
Memory trick: "**The milk is here, so the milk is off**" — forward. "**Is the milk off? Then check the date**" — backward. Facts push conclusions out; the goal pulls facts in.

Exam trap: Forward chaining can **never prove a negative**: no rule fires on "there is no rash", so "not measles" is never derived.

Worked example ✍️ trace both ways on the rules of page 2: Facts: temperature 39, cough true, white-cell count 7200, shortness of breath true. **Forward:** cycle 1 fires R1 → add *influenza 0.8*; cycle 2 fires R2 → add *pneumonia 0.9*; cycle 3 R3 fails. Result: pneumonia. **Backward:** goal = pneumonia → only R2 concludes it → it needs influenza → only R1 concludes that → it needs temperature, cough, white-cell count → **three questions to the user**, all yes → R1 proved → R2 satisfied. R3 is never looked at.

4 The Classic Systems & the Bottleneck

Definition: The **knowledge-acquisition bottleneck** is the hard part of any expert system: an expert's knowledge is tacit, verbal and full of unstated context, and turning it into explicit rules takes interviews, months of work and endless validation. The chain runs **domain expert ⇄ knowledge engineer ⇄ rule base**; nothing in it learns by itself.



every rule in every expert system passed through this loop by hand

System	Domain, reasoning and dates	Limitation / fate
DENDRAL 1965–1972	chemistry — infers a molecular structure from a mass spectrum, by splitting the molecule into substructures	first expert system; hours of computer time where a chemist needed minutes
MYCIN 1960s–77 EMYCIN 1977	medicine — bacteraemia diagnosis and therapy, ≈ 600 rules, backward chaining + certainty factors; EMYCIN is the same engine with those rules removed	≈ 65 % correct in blind tests, above the physicians who tested it; never used clinically — but EMYCIN proved the engine is reusable
XCON (R1) 1980	DEC — configures VAX-11/780 systems from a customer order, forward chaining; pilot 1978–80	one company, one product family; reported to save tens of millions of dollars a year

Worked example ✍️ a viva on MYCIN: **Q.** Why was MYCIN never used in hospitals? **A.** Because of its bottleneck and its sociology, not its accuracy: the rule base was elicited by hand from a few specialists, it knew nothing outside infectious disease, and liability, ethics and consent blocked a therapy-recommending machine.

Limitation	What it means
Knowledge acquisition	slow, costly, hand-made
Narrow domain	outside it, unanswerable
Brittleness	fails, never degrades
No learning	new facts = new interview
No common sense	no world model at all

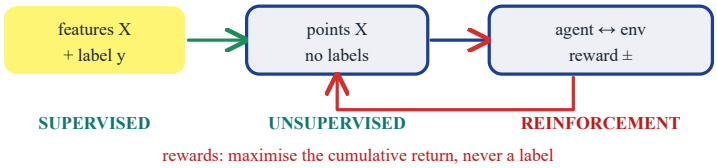
Exam trap: DENDRAL was **chemistry**, not medicine. MYCIN did **not** learn from patient records. XCON is not general purpose.

Memory trick: "**D**ENDRAL for **s**tructure, **M**YCIN for **s**ymptoms, XCON for a con**figuration**". All three are dead code today: the bottleneck, not the silicon, ended expert systems.

5 Machine Learning & Three Paradigms

Definition: **Machine learning** is the branch of AI in which a system improves its performance on a task by working through **data**, instead of having every rule written down in advance. Samuel used the term in **1959**; Mitchell's definition: a program learns from experience **E** with respect to a task **T**, measured by **P**, if its performance at **T** improves with **E**.

Term	Meaning	Term	Meaning
Data / example	one data row	Model	the learned f : X → y
Feature (X)	one input property	Parameter	a number the fit moves
Label (y)	the answer wanted	Hyper-parameter	a setting you choose
Training	fitting the parameters	Prediction	running the frozen model
Loss / cost	minimised while fitting	Epoch	one pass over training



three ways to get feedback — the label, the absence of a label, or a number after the act

Aspect	Supervised	Unsupervised	Reinforcement
Data	labelled (X, y)	unlabelled X	reward after acting
Goal	predict a known answer	discover structure	maximise the reward
Tasks	classification, regression	clustering, PCA	control, games, motion
Algorithms	tree, k-NN, naive Bayes, SVM	k-means, PCA, DBSCAN	Q-learning, policy gradient
Example	spam filter, credit scoring	market segmentation	a robot learning to walk

Memory trick: **"Labels in, prediction out. No labels, groups out. A number afterwards, behaviour out."** Ask of any algorithm only this: *where does the feedback come from?*

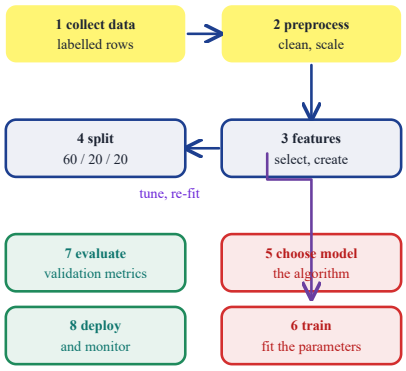
Worked example ✍ which paradigm?: (a) predict a house price → **regression, supervised**; (b) label an e-mail spam / not spam → **classification, supervised**; (c) split a million customers into segments → **clustering, unsupervised**; (d) cut 900 columns to 20 → **dimensionality reduction**; (e) a robot learning to walk from a per-step reward → **reinforcement**.

Exam trap: ML is a **subset** of AI; an expert system that never learns is still AI. k-means stays unsupervised even if you afterwards name the clusters.

6 Supervised Learning & the Process

Definition: Supervised learning fits a mapping **f : X → y** from labelled examples (X, y). **Classification** predicts a **discrete** label — spam / not spam, disease A / B / C. **Regression** predicts a **continuous** number — price, rainfall, demand. The label is known in the training data only.

Aspect	Classification	Regression
Output	one class from a finite set	a real number
Examples	spam filter, tumour type, fraud	house price, rainfall, demand
Error measured by	accuracy, precision, recall, F1	MAE, MSE, RMSE, R^2
Typical loss	cross-entropy	squared error
Typical models	tree, k-NN, naive Bayes, SVM	linear regression, tree regression
Answers	"which one?"	"how much?"



the loop candidates forget is step 7 → step 5

Eight steps — two are the ones candidates forget:

1. **Collect** labelled data — most of the work
2. **Preprocess** — missing values, duplicates
3. **Engineer features** — keep the signal
4. **Split** — shuffle, then **60 / 20 / 20**
5. **Choose a model** — start simple
6. **Train** — fit on training data only
7. **Evaluate** — k-fold CV beats one split
8. **Deploy and monitor** — data drifts

Worked example ✍ a mini-run: 1 000 labelled e-mails, 12 features. Split **600 / 200 / 200**. A depth-3 tree: **96 % train, 84 % validation**. A depth-12 tree: **100 % train, 79 % validation** — memorised noise, so keep depth 3. Open the test set **once**: 85 %. Quote **85 %**.

Memory trick: "**Collect, clean, features, split, fit, evaluate, deploy.**" The two steps everyone forgets — the **split** and the **monitor**.

Exam trap: The **test set is never tuned on**. Scaling **after** the split leaks test information. More trees is not automatically better: depth past the signal is overfitting.

IT link: Real projects spend most of their time in steps 1–3. A model is a one-line object; a clean labelled dataset with a defensible split is the actual asset.

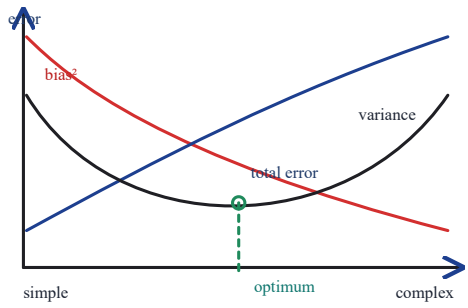
7 Split, Overfitting & Bias–Variance

Definition: Split the data: **training** to fit, **validation** to choose the model and its settings, **test** to report the final number once. A common split is **60 / 20 / 20**; **k-fold cross-validation** (k = 5 or 10) repeats the split k times so one lucky partition cannot decide the answer.

Set	Share	Used for	Never used for
Training	60 %	fitting the parameters	scoring the model
Validation	20 %	choosing model, hyper-parameters	fitting parameters
Test	20 %	the one final report	any choice at all

Model complexity	Train error	Validation error	Verdict
1 feature, linear	14 %	15 %	underfitting — high bias
depth-3 tree	4 %	6 %	just right — best choice
depth-20 tree + noise	0.5 %	22 %	overfitting — high variance

Underfitting & overfitting: **Overfitting**: the model fits the training data so closely that it memorises its noise — **training error ≪ test error**, high variance, no generalisation. **Underfitting**: too simple to capture the structure — **both errors high**, high bias. Expected test error = **bias² + variance + irreducible noise**; complexity trades the first two against each other.



total error = bias² + variance + irreducible noise

Worked example ✍️ choosing between two models:

Model A: bias² = 0.09, variance = 0.02, noise = 0.01 → total **0.12**. Model B: bias² = 0.01, variance = 0.16, noise = 0.01 → total **0.18**. A wins: B is the flexible model that memorised noise. More data shrinks B's **variance** but cannot shrink its bias.

Exam trap: **99 % training accuracy is not success** — a card that memorises the answers also gets 100 % on them. More data reduces **variance** only; reducing **bias** needs a more expressive model.

How overfitting is reduced: more and better data · regularisation (L1 / L2) · **k-fold cross-validation** · a simpler model · **pruning** a decision tree · **early stopping** · dropout · feature selection.

8 Evaluating a Model: Confusion Matrix

Definition: The **confusion matrix** counts what a classifier actually did, for a **binary** task: true positives, false positives, true negatives, false negatives. Every classification metric in the exam is one line of arithmetic on these four numbers.

Predicted positive	Predicted negative	
TP = **45** — flagged, truly bad	TN = **140** — cleared, truly good	185 correct
FN = **5** — missed, truly bad	FP = **10** — wrongly accused	15 wrong

The five numbers, one line each:

Accuracy = (TP + TN) / total

Precision = TP / (TP + FP)

Recall (sensitivity) = TP / (TP + FN)

Specificity = TN / (TN + FP)

F1 = 2 · P · R / (P + R)

MAE = $\Sigma|y - \hat{y}| / n$

For regression: RMSE = $\sqrt{(\Sigma(y - \hat{y})^2/n)}$ and $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$, which is 1 for a perfect fit and ≤ 0 for a model worse than predicting the mean.

Worked example ✍️ a fraud filter, 200 cases: TP = 45, FN = 5, TN = 140, FP = 10, total 200. Accuracy = 185/200 = **92.5 %** · Precision = 45/55 = **81.8 %** · Recall = 45/50 = **90 %** · Specificity = 140/150 = **93.3 %** · F1 = 2(0.818)(0.900)/(1.718) = **85.7 %**. Read the story: 92.5 % accuracy looks excellent, but the filter misses **5 real frauds** and harrasses **10 innocent customers**. Push the threshold and recall rises while precision falls.

Exam trap: Accuracy is the wrong single number on an **imbalanced** set: if 99 % of cases are negative, "always predict negative" scores 99 % and is useless. **Recall** is what a doctor wants; **precision** is what a bank wants before accusing someone.

Memory trick: **"Precision is how many of the things I accused were guilty; recall is how many of the guilty things I caught."** F1 is the harmonic mean, so it is only high when both are high.

IT link: These four numbers are what a model's report card contains, and what the ROC curve is built from (recall against 1 − specificity, swept over the decision threshold). The **threshold** is a business choice, not a modelling one.

9 Three Classic Learners, by Hand

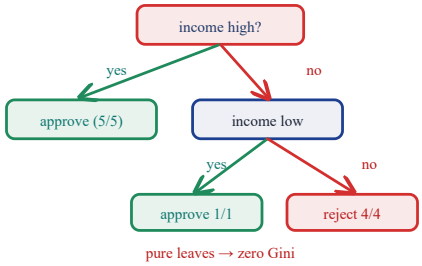
Definition: Three supervised algorithms that need no calculus and no software to score: the **decision tree** splits the data greedily to reduce impurity, **k-NN** copies the answer of the k nearest training points, and **naive Bayes** applies Bayes' theorem assuming the features are independent given the class.

Property	Decision tree	k-NN	Naive Bayes
Learns by	splits on impurity	storing the data	counting values
Predict cost	a few comparisons	distance to all points	one multiply / feature
Key knob	depth, leaf size	k, distance metric	Laplace smoothing
Weakness	overfits noisy tables	needs scaling, high dim.	independence assumed
Readiness	a flow of questions	none (lazy learner)	scale-independent

The two formulas you must be able to write:

Impurity. Gini $= 1 - \Sigma\, p_i^2$ · Entropy $H = -\Sigma\, p_i \log_2 p_i$. Split on the largest **information gain** $= H(\text{parent}) - \Sigma\, (n_i/n)\, H(\text{child}_i)$.

Naive Bayes. $P(c|x) \propto P(c) \cdot \prod_i P(x_i \mid c)$. Conditional independence *is* the assumption; **Laplace smoothing** stops an unseen value zeroing the product.



10 applicants, one tree, 100 % on the training table

Worked example 1 ✍ the tree: 10 applicants: 6 approved, 4 rejected. Root Gini $= 1 - (0.6^2 + 0.4^2) =$ **0.48**. Split on income → the 5 high-income are all approved (Gini 0), the 5 low-income are 1 approved and 4 rejected (Gini 0.32); weighted Gini = **0.16**. Split those five on credit score ≥ 700 → Gini 0. 10/10 on the training table, yet a high-income applicant with a terrible score would still be approved: **overfitting**.

Worked example 2 ✍ k-NN: The three nearest labelled neighbours of a new applicant in (income, score) space are **approved, approved, rejected**. With **k = 3** the majority vote is **approve**. With k = 1 you copy the nearest point and inherit its noise; with k = 200 you predict the file's majority class and nothing useful. k-NN is a **lazy learner**: it computes nothing at fit time.

Worked example 3 ✍ naive Bayes, 14 days of tennis weather: P(yes) = 9/14 and P(no) = 5/14. Query: sunny, hot, high humidity, strong wind. yes: $0.643 \times 0.222 \times 0.222 \times 0.333 \times 0.333 =$ **0.0035** · no: $0.357 \times 0.600 \times 0.600 \times 0.800 \times 0.625 =$ **0.0643** ⟹ **do not play**, roughly 18 : 1.

Memory trick: Tree = **ask questions**. k-NN = **ask the neighbours**. Naive Bayes = **multiply the odds** — and *naive* warns that the features are assumed independent.

Exam trap: k-NN is a **supervised** learner, not unsupervised — the labels exist, they are just not used at fit time. Entropy is **not** a probability.

Night-Before-the-Exam Sheet

Fact sheet:

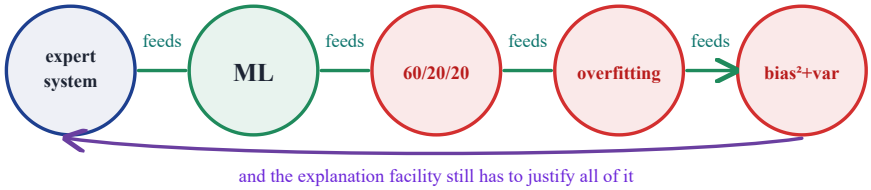
KB + IE + UI + EF = an ES	working memory = case facts
match → fire → update → test	conflict: specificity, recency
forward = data-driven, bottom-up	backward = goal-driven, top-down
DENDRAL 1965–1972, chemistry	MYCIN ≈ 600 rules, ≈ 65 %
EMYCIN 1977 · XCON from 1980	Samuel 1959 · Mitchell E-T-P
supervised / unsupervised / RL	split 60 / 20 / 20, k-fold CV
error = bias² + variance + noise	accuracy, precision, recall, F1

Top 12 one-liners for MCQs

1. The engine reasons; the base stores
2. Firing = copying the THEN into memory
3. Forward starts from facts, not the goal
4. Only backward chaining asks questions
5. DENDRAL = chemistry, MYCIN = medicine
6. An ES that never learns is still AI
7. Supervised needs labels, unsupervised not
8. The test set is opened once, at the end
9. 99 % train + 60 % test = overfitting
10. More data cuts variance, never bias
11. k-NN is supervised; naive Bayes = independence

Top traps: the engine does not store the rules · the explanation facility is not the user interface · forward chaining cannot prove a negative · DENDRAL is not a medical system · MYCIN never learned from patient records · XCON is not general purpose · accuracy is meaningless on an imbalanced set · precision ≠ recall · k-NN is supervised · entropy is not a probability.

All tricks together: "stores what it knows, works out what follows, explains why, asks you the rest" · "match, fire, update, test" · "the milk is here, so the milk is off" (forward) / "is the milk off? then check the date" (backward) · "D for structure, M for symptoms, X for configuration" · "labels in, prediction out; no labels, groups out; a number afterwards, behaviour out" · "collect, clean, features, split, fit, evaluate, deploy" · "train low, test high = overfitting; both high = underfitting" · "precision is how many I accused were guilty; recall is how many guilty I caught".



rules in, four conclusions out — and an explanation behind every one

✓ Practice MCQs (exam pattern, 10 marks)

1. Which part applies the rules?
(a) knowledge base (b) inference engine
(c) explanation box (d) user interface

2. MYCIN reasoned by:
(a) forward (b) k-means
(c) backward (d) reward learning

3. DENDRAL infers from a spectrum:
(a) a structure (b) a disease
(c) a configuration (d) a meaning

4. Forward chaining starts from the:
(a) goal hypothesis (b) reward signal
(c) training set (d) known facts

5. Supervised learning needs:
(a) unlabelled data (b) labelled pairs
(c) a reward per act (d) hand-written rules

6. 99 % train, 60 % test means:
(a) overfitting (b) underfitting
(c) high bias only (d) the optimum

7. k-NN with k = 1 predicts:
(a) nearest's class (b) mean of labels
(c) majority class (d) a random class

8. Naive Bayes assumes:
(a) separability (b) test distribution
(c) no missing data (d) independence

9. Recall equals:
(a) TP / (TP + FP) (b) TN / (TN + FN)
(c) TP / (TP + FN) (d) (TP+TN)/total

10. The test set reports:
(a) the parameters (b) the tuning
(c) the final result (d) the features

Answer key → 1-b · 2-c · 3-a · 4-d · 5-b · 6-a · 7-a · 8-d · 9-c · 10-c

Written question: Explain the architecture of an expert system with a labelled diagram, and forward vs backward chaining with examples. What is overfitting, how does the bias–variance tradeoff explain it, and how is it reduced? 10

1. **Architecture (3). Knowledge base** = the domain's facts, rules, heuristics. **Inference engine** = matches rules against working memory, fires them, returns the conclusions. **User interface** = asks the questions. **Explanation facility** = prints why. Cycle: **match → fire → update → test**. 3
2. **Forward chaining (2).** Data-driven, bottom-up: start from the facts, fire every rule whose antecedent matches, add each conclusion, repeat until no rule fires. Breadth-first; monitoring and prediction; XCON. 2
3. **Backward chaining (2).** Goal-driven, top-down: start from a hypothesis, find the rule concluding it, recurse on its conditions, ask the user for what is missing. Depth-first; diagnosis; MYCIN. 2
4. **Overfitting and bias–variance (3).** The model fits the training data so closely that it memorises noise, so **training error ≪ test error** — high variance. Expected test error = **bias² + variance + noise**. Remedies: more data, regularisation, k-fold CV, a simpler model, pruning, early stopping, dropout. 3

Before the exam ✓

- ✓ Can draw the architecture with its cycle
- ✓ Can run one forward-chaining cycle
- ✓ Can name DENDRAL, MYCIN, XCON with dates
- ✓ Can compute accuracy, recall and F1